

CM2005 OOP – Midterms 2024

Student Name: Devon Maya Xavier
UOL Student ID: 200618744
SIM Student ID: 10204322
Email: dmxavior001@mymail.sim.edu.sg

TASK 1: Compute candlestick data

The objective of this task was to compute candlestick data using the temperature data provided for a specific European country, spanning the years 1980 to 2019 — a total of 40 years of weather data.

To begin, the raw data was extracted from a CSV file (which I renamed `weather_data.csv` for convenience). This was accomplished using the `loadData` function, which is part of the `weatherManager` class I developed. The `weatherManager` class plays a key role in handling weather data, making it easier to filter, process, and manage the data throughout the project. It handles the task of loading, filtering, and performing subsequent operations like predictions, as required by the project's objectives.

```
src > weatherManager.cpp > ...
1  #include "weatherManager.h"
2  #include <fstream>
3  #include <sstream>
4  #include <map>
5  #include <set>
6  #include <algorithm>
7  #include <numeric>
8  #include <iostream>
9
10 // To store column headers for identifying country codes
11 std::vector<std::string> headers;
12
13 // -----
14 // 1) Load raw weather data from CSV
15 // -----
16 void weatherManager::loadData(const std::string& filePath) {
17     dataFilePath = filePath; // Save the file path
18     std::ifstream file(filePath);
19     if (!file) {
20         std::cerr << "Error: Could not open file " << filePath << std::endl;
21         return;
22     }
```

To generate the candlestick data, I created a Candlestick class to handle the computation of the required data. The logic for this computation is encapsulated in the candlestickData.h and candlestickData.cpp files, which were specifically developed to manage the processing and generation of the candlestick data.

```
src > C candlestickData.h > ...
1  #ifndef CANDLESTICKDATA_H
2  #define CANDLESTICKDATA_H
3
4  #include <string>
5  #include <iostream>
6
7  // Candlestick Class
8  class Candlestick {
9  private:
10     std::string date; // could be "YYYY", "YYYY-MM", or full "YYYY-MM-DD"
11     double open; // average mean temperature of the previous time frame
12     double high; // highest temperature in the current time frame
13     double low; // lowest temperature in the current time frame
14     double close; // average mean temperature of the current time frame
15
16 public:
17     // Constructor to initialize candlestick data
18     Candlestick(const std::string& date, double open, double high, double low, double close)
19         : date(date), open(open), high(high), low(low), close(close) {}
20
```

```
src > C candlestickData.cpp > ...
1  #include "candlestickData.h"
2  #include "weatherManager.h"
3  #include <iostream>
4  #include <vector>
5  #include <numeric>
6  #include <algorithm>
7  #include <map>
8  #include <string>
9
10 // Helper function to extract the year from a timestamp (assuming format "YYYY-MM-DD HH:MM:SS")
11 std::string extractYear(const std::string& timestamp) {
12     return timestamp.substr(0, 4); // First 4 characters represent the year
13 }
14
15 extern weatherManager manager; // Declare manager as extern
16
17 void computeCandlestickData() {
18     std::cout << "Computing candlestick data...\n";
19
20     // Get weather data from manager
21     std::vector<WeatherData> weatherData = manager.getWeatherData();
22
23     std::map<std::string, std::vector<double>> yearlyData; // Map to group temperatures by year
24
25     // Group temperatures by year
```

To test the computation of the candlestick data (i.e., open, close, high, and low values), I decided to generate yearly candlestick data, further filtered by global or country-specific datasets, to ensure the function works correctly.

The results are as follows:

```
Weather Analysis Toolkit
=====
Please choose an option from the menu below:
1. Load Data - Please load data to proceed with below options
2. Compute Candlestick Data
3. Create Text-Based Candlestick Plot
4. Filter Data
5. Predict Weather Data
6. Help
7. Exit
Enter your choice: 1
Enter the path to the weather data file: weather_data.csv
Data loaded successfully from weather_data.csv. Total rows: 350640
Timestamp Preview: 1980-01-01T00:00:00Z, Temperatures: -3.64 -0.72 4.664 -6.287 -3.422 -1.261 -1.87 -7.06 8.066 -6.254 -0.537 -1.249 9.912 -0.88 -3
.021 1.292 0.301 -6.541 -5.15 -7.166 2.382 -7.038 -3.721 12.862 -0.031 -3.945 -3.055 -4.648
Timestamp Preview: 1980-01-01T01:00:00Z, Temperatures: -3.803 -1.165 4.052 -6.602 -3.36 -1.414 -1.914 -7.341 7.96 -6.232 -0.489 -1.377 9.369 -0.888
-2.793 1.121 0.145 -6.509 -5.333 -7.371 2.236 -6.941 -3.806 12.757 0.311 -4.053 -3.272 -4.554
Timestamp Preview: 1980-01-01T02:00:00Z, Temperatures: -3.969 -1.434 3.581 -6.981 -3.429 -1.571 -1.976 -7.591 8.008 -6.343 -0.391 -1.449 9.056 -0.9
5 -2.597 0.98 -0.029 -6.416 -5.167 -7.342 2.086 -6.856 -3.868 12.674 0.568 -4.129 -3.639 -4.455
Timestamp Preview: 1980-01-01T03:00:00Z, Temperatures: -4.076 -1.843 3.197 -7.368 -3.697 -1.76 -2.04 -7.716 8.065 -6.55 -0.354 -1.481 8.658 -1.157
-2.568 0.944 -0.191 -6.197 -5.105 -7.206 1.861 -6.761 -3.934 12.622 0.61 -4.139 -4.202 -4.421
Timestamp Preview: 1980-01-01T04:00:00Z, Temperatures: -4.248 -2.094 2.75 -7.849 -4.081 -1.995 -2.106 -7.638 8.009 -6.722 -0.354 -1.433 8.335 -1.46
8 -2.689 0.91 -0.275 -5.899 -5.218 -7.073 1.713 -6.569 -4.064 12.522 0.462 -4.239 -4.732 -4.449
=====
Weather Analysis Toolkit
```

To proceed with the rest of the options, the user must first select option 1 from the menu to load a data file. The prompt to enter the path to the weather file allows flexibility, enabling the application to work with different data sets, as long as the files are stored in the same directory and use the same parameters (or tokens). Once the file is loaded, a preview of the first 5 rows of the data will be displayed, confirming that the correct data has been retrieved.

```
// -----
// 1) Load raw weather data from CSV
// -----
void weatherManager::loadData(const std::string& filePath) {
    dataFilePath = filePath; // Save the file path
    std::ifstream file(filePath);
    if (!file) {
        std::cerr << "Error: Could not open file " << filePath << std::endl;
        return;
    }

    std::string line;
    bool isFirstLine = true;

    while (std::getline(file, line)) {
        std::stringstream ss(line);
        std::string value;

        if (isFirstLine) {
            // Extract headers from the first line
            std::getline(ss, value, ','); // Skip "utc_timestamp"
            while (std::getline(ss, value, ',')) {
                headers.push_back(value);
            }
        }
    }
}
```

```
// -----
// 2) Access the loaded weather data
// -----
const std::vector<WeatherData>& weatherManager::getWeatherData() const {
    return weatherData;
}

// -----
// 3) Show a preview of the first 'n' rows of raw data
// -----
void weatherManager::showPreview(size_t rows) const {
    size_t count = 0;
    for (const auto& data : weatherData) {
        std::cout << "Timestamp Preview: " << data.timestamp << ", Temperatures: ";
        for (double temp : data.temperatures) {
            std::cout << temp << " ";
        }
        std::cout << std::endl;

        if (++count >= rows) break;
    }
}
```

```
=====
Please choose an option from the menu below:
1. Load Data - Please load data to proceed with below options
2. Compute Candlestick Data
3. Create Text-Based Candlestick Plot
4. Filter Data
5. Predict Weather Data
6. Help
7. Exit
Enter your choice: 2
Would you like to see data for:
1. Global (all countries)
2. A specific country
Enter your choice (1 or 2): 2
Available country codes from the data file:
AT_temperature
BE_temperature
BG_temperature
CH_temperature
CZ_temperature
DE_temperature
DK_temperature
EE_temperature
ES_temperature
FI_temperature
FR_temperature
GB_temperature
GR_temperature
HR_temperature
HU_temperature
IE_temperature
IT_temperature
LT_temperature
LU_temperature
LV_temperature
NL_temperature
NO_temperature
PL_temperature
PT_temperature
RO_temperature
SE_temperature
SI_temperature
SK_temperature
Enter the country code to analyze: █
```

Next, the user proceeds to Option 2 to compute the candlestick data. They are given the option to choose between global or country-specific yearly candlestick data. A list of available countries is displayed for the user to select from. After choosing their preferred country, the user is shown the number of candlestick data generated (in this case, 40, corresponding to the 40 years of data for each country). The user is then prompted to decide if they want to view the generated data. All of these options are managed by the menu.cpp, menu.h files, the computeCandlestickData function, and the weatherManager class. This successfully achieves the goal of Task 1.

```
// -----
// 4) Get Unique Country Codes from Headers
// -----
std::set<std::string> weatherManager::getCountryCodes() const {
    std::set<std::string> countryCodes;

    std::ifstream file(dataFilePath);
    if (!file.is_open()) {
        std::cerr << "Error: Unable to open the file for extracting headers." << std::endl;
        return countryCodes;
    }

    std::string line;
    if (std::getline(file, line)) {
        std::stringstream ss(line);
        std::string header;
```

```
// -----
// 7) Compute Candlestick Data
// -----
void weatherManager::computeCandlestickData(const std::string& selectedCountryCode) {
    candlestickData.clear();
    std::map<int, std::vector<double>> yearToTemps;

    // Find the index of the selected country code
    auto it = std::find(headers.begin(), headers.end(), selectedCountryCode);
    if (it == headers.end() && selectedCountryCode != "global") {
        std::cerr << "Error: Invalid country code '" << selectedCountryCode << "' provided.\n";
        return;
    }

    size_t columnIndex = std::distance(headers.begin(), it);
```

```
Enter the country code to analyze: AT_temperature
Candlestick data computed for AT_temperature. Total candlesticks: 40
Would you like to view the computed candlestick data now? (y/n): y
----- Candlestick Details -----
Date: 1980, Open: -3.64, High: 29.132, Low: -14.507, Close: 6.04915
Date: 1981, Open: 6.04915, High: 29.419, Low: -16.219, Close: 7.19199
Date: 1982, Open: 7.19199, High: 28.395, Low: -15.084, Close: 7.56654
Date: 1983, Open: 7.56654, High: 32.416, Low: -18.098, Close: 8.05365
Date: 1984, Open: 8.05365, High: 32.659, Low: -13.338, Close: 6.83589
Date: 1985, Open: 6.83589, High: 29.166, Low: -22.705, Close: 6.57698
Date: 1986, Open: 6.57698, High: 29.785, Low: -20.601, Close: 7.12091
Date: 1987, Open: 7.12091, High: 28.054, Low: -25.301, Close: 6.5998
Date: 1988, Open: 6.5998, High: 31.213, Low: -13.010, Close: 7.6885
```

TASK 2: Create a text-based plot of the candlestick data

For task 2, the objective was to create a text-based plot of the candlestick data (i.e., yearly in my case). The goal was to represent the candlestick data visually using text output, mimicking a graphical plot.

To begin, I first decided on the characters to be used to represent various elements of the candlestick chart.

Here is the approach I followed for this task:

1. Characters for the Candlestick Plot:

- I used | to represent the stalk of the candlestick, which is the vertical line connecting the high and low temperatures.
- The + character was used to represent the high temperature for each candlestick.
- The - character was used to represent the low temperature for each candlestick.
- The O and C characters were used to represent the open and close temperatures, respectively. The open and close markers would be placed at the respective row positions of the plot where they occurred.

```
int updatedCandlestickPadding = candlestickPadding;
// Plot High to Low with vertical lines
if (row <= highRow && row >= lowRow) {
    if (row == openRow && row == closeRow) {
        std::cout << "OC"; // Both Open and Close on the same row
        updatedCandlestickPadding -= 1;
    } else if (row == openRow) {
        std::cout << "O";
    } else if (row == closeRow) {
        std::cout << "C";
    } else if (row == highRow) {
        std::cout << "+";
    } else if (row == lowRow) {
        std::cout << "-";
    } else {
        std::cout << "|"; // Candlestick body
    }
} else {
    std::cout << " ";
}

// Add padding after each candlestick
for (int p = updatedCandlestickPadding; p > 0; --p) {
    std::cout << " ";
}
```

2. Designing the Plot:

- The plot was structured as a vertical graph with the temperature range on the y-axis and the years on the x-axis.
- The **y-axis** was represented by the temperature scale, ranging from -40°C to 40°C (or whatever the defined min/max temperature range was).
- The **x-axis** showed the years for which the data was generated.

```
// Function to print the ASCII candlestick graph
void printCandlestickGraph(const std::vector<Candlestick>& filteredData) {
    const double minTemp = -40.0, maxTemp = 40.0;
    const int rows = 20; // Number of rows for graphing
    const int candlestickPadding = 5; // Padding between candlesticks (adjust this value as needed)

    // Print the graph header
    std::cout << "\nCandlestick Graph for Selected Range:\n";
    std::cout << "Temperature (°C)\n";

    // Print the legend
    std::cout << "\nLegend: ";
    std::cout << "O = Open, C = Close, + = High, - = Low, | = Candlestick Body\n";
    std::cout << "-----\n";

    // Loop through each row (y-axis)
    for (int row = rows; row >= 0; --row) {
        double temp = minTemp + row * (maxTemp - minTemp) / rows;
        std::cout << std::setw(3) << static_cast<int>(temp) << " | "; // Print temperature on y-axis with 3 characters

        bool previousColumnHasCharacter = false; // Flag to track previous column's character

        // Print x-axis (Years)
        std::cout << "      +";
        for (size_t i = 0; i < filteredData.size(); ++i) {
            std::cout << "-----+"; // Adjust the width of the spaces between each year
        }
        std::cout << "\n      "; // Adjust alignment to match the candlesticks

        // Loop through the filtered data and print the year labels with additional spacing
        for (size_t i = 0; i < filteredData.size(); ++i) {
            std::string year = filteredData[i].getDate().substr(2); // Get last 2 digits of the year
            if (i == 0) {
                // Add extra space before the first year
                std::cout << " ";
            }
            std::cout << std::setw(4) << year << " "; // Print the year with 4 spaces, plus 2 extra spaces
        }
        std::cout << "\n";
    }
}
```

3. Implementation:

- First, the `normalizeTemperature` function was used to map the temperature values to the corresponding rows of the plot.
- For each row of the plot, the data was checked for whether the high, low, open, and close values fell within that row's temperature range, and the appropriate character (|, +, -, O, C) was printed.

TASK 3: Filter Data and Plotting using text

For Task 3, the goal was to provide filtering options for the data and plot the filtered data using a text-based graph. I implemented three types of filters:

- **Filter by Country**
- **Filter by Date Range**
- **Compute Averages**

To represent the filtered data, I chose to use a bar graph, which is specifically applied to the average values to illustrate the trend in average temperatures. For this task, I decided not to apply graph plotting to the first two filters (country and date range), as the output size would be too large. The bar graph effectively displays the daily, monthly, or yearly trends for both global and country-specific temperatures within a selected date range.

The first step involved implementing filtering mechanisms for each of these options in the filterData.h, filterData.cpp, and weatherManager class files. After applying the filters, the filtered data was plotted using bar graphs, with the plotting logic stored in plotData.cpp. This approach enabled clear visualization of the trends based on the selected criteria, allowing users to easily observe temperature patterns according to their chosen filters.

```
void filterData() {
    std::cout << "Filtering data by country...\n";

    // Display available country codes
    std::set<std::string> countryCodes = manager.getCountryCodes();
    if (countryCodes.empty()) {
        std::cout << "No country codes available. Please ensure data is loaded.\n";
        return;
    }

    std::cout << "Available country codes:\n";
    for (const auto& code : countryCodes) {
        std::cout << code << "\n";
    }

    // Ask the user to select a country code
    std::string selectedCountryCode;
    std::cout << "Enter the country code to filter by: ";
    std::cin >> selectedCountryCode;
```

Approach:

1. Filtering Raw data by Country:

The first filtering option allows the user to select a specific country (e.g., AT_temperature, FR_temperature, etc.) from the available options. Once a country is chosen, only the data corresponding to that country is displayed. To achieve this, the getRawWeatherDataByCountry function was used to filter the data based on the selected country code. The filtered data was then printed to the terminal for the user to review.

```
// -----
// 5) Filter Raw Weather Data by Country
// -----
std::vector<WeatherData> weatherManager::getRawWeatherDataByCountry(const std::string& countryCode) const {
    std::vector<WeatherData> filteredData;

    // Check if the country code exists in the headers
    auto it = std::find(headers.begin(), headers.end(), countryCode);
    if (it == headers.end()) {
        std::cerr << "Error: Invalid country code '" << countryCode << "' provided.\n";
        return filteredData;
    }

    size_t countryIndex = std::distance(headers.begin(), it);

    // Filter data for the selected country
    for (const auto& data : weatherData) {
        if (countryIndex < data.temperatures.size()) {
            WeatherData filteredRow = data;
        }
    }
}
```

2. Filtering by Date Range:

The second filtering option allows the user to specify a date range for the data. For instance, if a user wants data only from 1980 to 1990, the program filters out any data outside that range.

To implement this, I used the `getRawWeatherDataByDateRange` function in the `weatherManager` class. This function filters the raw data by parsing the year from the timestamp and comparing it to the user-defined start and end years. This ensures that only data within the specified date range is included.

```
// -----
// 6) Filter Raw Weather Data by Date Range
// -----
std::vector<WeatherData> weatherManager::getRawWeatherDataByDateRange(const std::string& startDate, const std::string& endDate) const {
    std::vector<WeatherData> filteredData;

    // Iterate over the weather data
    for (const auto& data : weatherData) {
        if (data.timestamp >= startDate && data.timestamp <= endDate) {
            filteredData.push_back(data); // Add rows within the date range
        }
    }

    return filteredData;
}
```

3. Compute Averages with Filters:

The third filtering option allows the user to compute averages for daily, monthly, or yearly temperatures, either for global data or specific countries. To implement this, I utilized a helper function to categorize the data into daily, monthly, or yearly segments. Additionally, a simple function was used to calculate the averages for the selected filtered data, enabling the user to view the temperature trends over the chosen time periods.

```

std::map<std::string, double> averages;
for (const auto& [key, temps] : groupedTemps) {
    if (!temps.empty()) {
        averages[key] = std::accumulate(temps.begin(), temps.end(), 0.0) / temps.size();
        std::cout << "Average for " << key << " = " << averages[key] << "\n";
    }
}

```

```

// -----
// 11) Helper for Averages
// -----
std::string extractDatePart(const std::string& timestamp, const std::string& granularity) {
    if (granularity == "day" || granularity == "daily") {
        if (timestamp.size() >= 10) return timestamp.substr(0, 10); // YYYY-MM-DD
    }
    if (granularity == "month" || granularity == "monthly") {
        if (timestamp.size() >= 7) return timestamp.substr(0, 7); // YYYY-MM
    }
    if (granularity == "year" || granularity == "yearly") {
        if (timestamp.size() >= 4) return timestamp.substr(0, 4); // YYYY
    }
    std::cerr << "Error: Invalid timestamp or granularity: " << timestamp << ", " << granularity << "\n";
    return "";
}

```

The output:

The output of the averages is presented using bar graphs, where the * symbol represents the values. However, negative values are not displayed as part of the bar graph; instead, they are printed in brackets. This is an area that requires further refinement. With that, Task 3 has been completed.

```

1. Daily
2. Monthly
3. Yearly
Enter your choice: 3
Enter the start date (YYYY-MM-DD): 1980
Enter the end date (YYYY-MM-DD): 1986
Processing weather data for country code 'AT_temperature' between '1980' and '1986'
Average for 1980 = 6.05
Average for 1981 = 7.19
Average for 1982 = 7.57
Average for 1983 = 8.05
Average for 1984 = 6.84
Average for 1985 = 6.58
Averages computed successfully. Plotting...

Graph (Bar Chart):
1980 | ##### (6.05)
1981 | ##### (7.19)
1982 | ##### (7.57)
1983 | ##### (8.05)
1984 | ##### (6.84)
1985 | ##### (6.58)

```

TASK 4: Predicting Data and Plotting

For Task 4, the objective was to predict temperature changes over a selected date range for a specified country. I chose to implement a Linear Regression model for this prediction. The prediction functionality is implemented in the predictData.h and predictData.cpp files, and also managed by the weatherManager class. This model utilizes historical monthly temperature data, fitting a linear equation to the data, and then predicts the average temperature for a given year within the specified range.

Linear regression was selected for this task because of its simplicity and effectiveness in modelling the relationship between time and temperature. The model assumes a linear relationship between the two variables, which is well-suited for temperature data. Since the data follows a time-series pattern, where temperature changes over time tend to exhibit a relatively predictable trend, linear regression is an ideal choice for this type of forecasting.

```
> G predictData.cpp > ...
#include "predictData.h"
#include <numeric>
#include <cmath>

extern weatherManager manager; // Access global weatherManager instance

// Helper function to calculate historical monthly averages
std::vector<double> calculateHistoricalMonthlyAverages(const std::string& countryCode, int month) {
    const auto& weatherData = manager.getRawWeatherDataByCountry(countryCode);
    std::vector<double> monthlyAverages;

    // Loop through historical data
    for (int year = 1980; year <= 2019; ++year) {
        std::vector<double> temperatures;

        // Extract temperatures for the specified month and year
        for (const auto& data : weatherData) {
            if (std::stoi(data.timestamp.substr(0, 4)) == year &&
                std::stoi(data.timestamp.substr(5, 2)) == month) {
                temperatures.push_back(data.temperatures.front());
            }
        }
    }
}

// Linear regression prediction for a single month
double predictMonthlyTemperature(const std::vector<double>& historicalAverages, int predictionYear) {
    int startYear = 1980;
    int endYear = 2019;

    // Prepare x (years) and y (average temperatures)
    std::vector<int> years;
    for (int i = startYear; i <= endYear; ++i) {
        years.push_back(i);
    }

    // Linear regression calculations
    int n = years.size();
    double sumX = std::accumulate(years.begin(), years.end(), 0.0);
    double sumY = std::accumulate(historicalAverages.begin(), historicalAverages.end(), 0.0);
    double sumXY = 0.0;
    double sumX2 = 0.0;
```

```

// Function to get predicted temperatures for the specified months and year
std::vector<double> getPredictedTemperatures(const std::string& countryCode, int startMonth, int endMonth, int predictionYear) {
    std::vector<double> predictedTemps;

    // Loop over the months in the specified range
    for (int month = startMonth; month <= endMonth; ++month) {
        // Calculate the historical monthly averages for the specified month
        auto monthlyAverages = calculateHistoricalMonthlyAverages(countryCode, month);

        // Use the linear regression model to predict the temperature for the given month and year
        double predictedTemp = predictMonthlyTemperature(monthlyAverages, predictionYear);

        predictedTemps.push_back(predictedTemp); // Store the predicted temperature for this month
    }

    return predictedTemps; // Return the list of predicted temperatures
}

```

The function uses the monthly averages calculated from historical data and then runs them with the linear regression model to generate predicted average monthly temperature for the selected year. A formula is then applied to calculate the temperature difference for the monthly averages computed prior. The results are then plotted in a bar graph as well for better visualisation. This completes Task 4.

```

SI_temperature
SK_temperature
Enter the country code for prediction (e.g., AT_temperature): AT_temperature
Enter the start month for prediction (1-12): 1
Enter the end month for prediction (1-12): 6
Enter the prediction year (e.g., 2022): 2025
Predicted temperature difference between 1 and 6 in 2025: 20.23°C

Predicted average temperatures (1-6):
1 | * (-2.0°C)
2 | * (-0.8°C)
3 | ** (3.3°C)
4 | *** (9.4°C)
5 | **** (13.2°C)
6 | ***** (18.2°C)

Temperature difference between month 1 and month 6 is: 20.2°C

Average temperature for the selected range (1-6): 6.9°C
Prediction complete.
=====

```